Milestone: 20GB Reached

We’ve reached 20GB of stored data — thank you for helping us grow!

Patreon Ko-fi Liberapay
Close
Freedium

- ko-fi
- librepay
- patreon

< Go to the original

Feature	LangGraph Platform	Aegra
Cost	$$$+ per month	Free (self-hosted)
Data Control	Third-party hosted	Your infrastructure
Vendor Lock-in	High dependency	Zero lock-in
Authentication	Limited options	Custom (JWT/OAuth/Firebase)
Database	Managed, no BYO	Bring your own Postgres
Tracing	LangSmith only	Your choice (Langfuse, etc.)
SDK Compatibility	LangGraph SDK	Same LangGraph SDK

Can't self-host LangGraph? Skill issue (and the fix)

You build a LangGraph workflow, it works on your laptop, and then production shows up with streaming, thread persistence, auth...

Agent Native Follow
~9 min read · February 18, 2026 (Updated: February 18, 2026) · Free: No

You build a LangGraph workflow, it works on your laptop, and then production shows up with **streaming, thread persistence, auth, multi-tenant filtering, migrations, observability.**

Teams (and most probably you) treat **agent infrastructure** like it's just an API wrapper. It isn't, and if you pick the wrong platform, you'll wake up one day realizing your tracing, state, and auth are welded to someone else's product roadmap.

Aegra is interesting because it doesn't try to be a new agent framework.

It tries to be the boring thing we all need: **a self-hosted, drop-in backend compatible with the LangGraph SDK**, with **PostgreSQL persistence**, **configurable auth**, and **OpenTelemetry fan-out**.

Feature	LangGraph Platform	Aegra
Cost	$$$+ per month	Free (self-hosted)
Data Control	Third-party hosted	Your infrastructure
Vendor Lock-in	High dependency	Zero lock-in
Authentication	Limited options	Custom (JWT/OAuth/Firebase)
Database	Managed, no BYO	Bring your own Postgres
Tracing	LangSmith only	Your choice (Langfuse, etc.)
SDK Compatibility	LangGraph SDK	Same LangGraph SDK

And it contains a couple of quietly smart engineering decisions I wish more **agent platforms** copied like caching compiled graphs without sharing execution instances, and running DB migrations automatically on startup.

The catch is self-hosting means you inherit the sharp edges too like the real-world pain of a Postgres major-version upgrade and the discipline required to not accidentally ship unauthenticated custom routes.

We'll get practical about that, with config snippets, code, and the tradeoffs I'd want to see in a design review.

> I'm writing a deep-dive ebook on **Agentic SaaS,** the emerging design patterns that are quietly powering the most innovative startups of 2026.
>
> **You can grab the first chapter here:** *Agentic SaaS Patterns Winning in 2026*, packed with real-world examples, architectures, and workflows you won't find anywhere else.

Why are we talking about this now?

Agents escaped the lab.

Not because they're suddenly "AGI," but because the shape of product work changed:

- Teams are shipping **LLM features** that need **stateful workflows** (multi-step, branching, tool calls, approvals).
- The cost and compliance pressure is real: **where you host**, **where you store**, and **how you observe** your agent runs is no longer optional.

And the ecosystem matured enough that we now have:

- workflow engines (LangGraph)
- UIs (Agent Chat UI / LangGraph Studio / CopilotKit-style frontends)
- and emerging standards like an **Agent Protocol** for interoperable agent APIs

So the conversation shifted from "How do I make an agent?" to:

> *"How do I run 10,000 agent runs a day without losing state, losing traces, or losing control?"*

Alternatives like Aegra exist precisely in that gap, it aims to be **LangGraph Platform-compatible**, but self-hosted, so you can keep your infra and governance model.

Aegra in one sentence

Aegra is a **self-hosted LangGraph Platform alternative.** It's designed as a **drop-in replacement**, using the **same LangGraph SDK and APIs**, but running on **your infrastructure with PostgreSQL persistence**.

If you already have LangGraph graphs, the pitch is basically:

- don't rewrite your agent logic
- don't rewrite the backend plumbing either
- run the backend yourself

What Aegra actually includes

When people say "agent backend," here's what they usually mean in practice:

Concern	What it becomes in production
Execution	Runs that can stream intermediate steps and survive network hiccups
State	Threads/checkpoints so the workflow is resumable and inspectable
AuthZ	Multi-tenant guardrails (filter runs/threads, inject metadata, deny actions)
Observability	Traces you can use for debugging and evaluation, not just vanity dashboards
Extensibility	Custom endpoints (webhooks, admin ops, internal integrations)
Persistence	Durable store for conversations + optional semantic memory

Aegra supports **human-in-the-loop gates**, **streaming**, **persistent state via PostgreSQL checkpoints**, **configurable auth**, **unified OTEL observability**, a **semantic store with pgvector**, and **custom routes via FastAPI**.

You can keep your existing LangGraph client code

You can use the LangGraph SDK client, point it at your server, create an assistant, create a thread, and stream a run.

Copy

```
from langgraph_sdk import get_client

client = get_client(url="http://localhost:8000")

assistant = await client.assistants.create(graph_id="agent")
thread = await client.threads.create()

async for chunk in client.runs.stream(
    thread_id=thread["thread_id"],
    assistant_id=assistant["assistant_id"],
    input={"messages": [{"type": "human", "content": "Hello!"}]},
):
    print(chunk)
```

That's the big unlock as you're not adopting a new SDK.

And Aegra leans hard on a CLI-driven dev workflow, `aegra dev` starts PostgreSQL and runs the server with hot reload.

You get boring enterprise knobs without giving up the agent ergonomics

Aegra's config model covers the knobs teams actually need:

- graph registration (`graphs`)
- auth hook (`auth.path`)
- custom FastAPI app mounting (`http.app`)
- CORS
- semantic store indexing (`store.index`)

- dependencies path injection for shared modules (`dependencies`)

It also supports configuration resolution order via an env var override, which matters when you have multiple environments:

1. `AEGRA_CONFIG` env var
2. `aegra.json`
3. `langgraph.json` (fallback for compatibility)

That's a small thing… until you're debugging why staging loaded the wrong config.

Critical parts

Here are the non-obvious points from the project that I think matter most if you're building an AI product that has to survive contact with reality.

Cache the compiled structure

If you've ever tried to run a shared in-memory graph instance concurrently, you know where this goes: locks, races, and eventually the dreaded "we'll just scale vertically and pray."

Aegra's has a pattern I like a lot:

- cache the **base compiled graph** (immutable)
- for each request, create a **fresh copy** and inject checkpointer/store
- do it via a context manager so the lifecycle is explicit

```
async with langgraph_service.get_graph(graph_id) as graph:
    async for event in graph.astream(input, config):
        ...
```

Copy

Why I think this is underrated:

- You don't need locks if what you cache is immutable.
- You get isolation per execution without paying the cost of recompiling the graph every time.
- Your runtime concerns (checkpoints, store, tenancy context) become injected dependencies, not global state.

Tradeoff is fresh copies per request can mean more per-request overhead than a shared instance. But the alternative is hidden complexity and concurrency bugs, so pick your poison.

Auto-migrations on startup are a productivity superpower

Aegra automatically applies pending Alembic migrations on server startup (`alembic upgrade head`).

That's a huge contributor to a smooth dev workflow:

- new engineer pulls repo
- runs `aegra dev`
- DB schema is correct without "did you run migrations?" folklore

But in production, auto-migrations deserve respect.

It's good because it reduces "schema drift" incidents, and makes deploys more deterministic (migration happens as part of boot)

But it's risky because you're coupling deploy success to migration success, long-running migrations can extend downtime and rollback stories get messy (code rollback doesn't roll back DB changes automatically)

You have to tighten security and remove dev conveniences (no reload, no code mounts, secrets management, internal networks).

So auto-migrate is fine if you also do:

- backups
- migration review discipline
- staging rehearsal
- and you understand what happens on failed startup

If that sounds like a lot… yes. That's self-hosting.

Observability should be pure OTEL

Aegra describes a **Unified OpenTelemetry architecture** with the ability to fan-out traces to multiple backends simultaneously.

Config is env-driven and explicit:

```
OTEL_SERVICE_NAME=aegra-backend
OTEL_TARGETS="LANGFUSE,PHOENIX"   # or "GENERIC" or ""
OTEL_CONSOLE_EXPORT=false
```

Copy

It also calls out target-specific variables (Langfuse keys, Phoenix collector endpoint, generic OTLP endpoint/headers).

This means:

- You can send traces to a hosted tool and a local debugger at the same time.
- You can change your tracing backend without changing your agent code.
- OTEL becomes the "contract," not any single vendor's SDK.

However OTEL can be non-trivial to operate well. If you enable everything with no sampling or thought, you can absolutely drown yourself in trace volume and cost.

Semantic memory is configuration

Aegra supports a semantic store via the LangGraph Store API using PostgreSQL + pgvector. You configure an index with:

- embedding dimensions (`dims`)
- embedding provider/model (`embed`)

- optional fields selection (`fields`)

```
{
  "graphs": {
    "agent": "./graphs/react_agent/graph.py:graph"
  },
  "store": {
    "index": {
      "dims": 1536,
      "embed": "openai:text-embedding-3-small",
      "fields": ["$"]
    }
  }
}
```

Then you can store preferences and query by meaning:

```
from langgraph_sdk import get_client

client = get_client(url="http://localhost:8000")

await client.store.put(
    namespace=["user", "123", "preferences"],
    key="coding_style",
    value={"text": "I prefer clean code with descriptive variable names"}
)

results = await client.store.search(
    namespace_prefix=["user", "123"],
    query="How does this user like to write code?",
    limit=5
)
```

A few caveats:

- `dims` must match the embedding model output exactly (dimension mismatch errors are real).
- you need pgvector available (they recommend `pgvector/pgvector:pg18`).
- if you don't configure `store.index`, it falls back to basic key-value mode (backward compatible).
- you can embed only specific JSON fields (which matters for performance and privacy).

This is the right abstraction. Memory should be a storage + retrieval contract you can reason about and test.

Auth is a policy layer

Aegra's auth model is simple and powerful:

- you define an auth object using the LangGraph SDK `Auth()`
- you implement `@auth.authenticate` to produce a user identity + metadata
- you optionally define authorization handlers (`@auth.on.*`) for fine-grained policies

Here's a minimal auth file:

```
from langgraph_sdk import Auth

auth = Auth()

@auth.authenticate
async def authenticate(headers: dict) -> dict:
    token = headers.get("Authorization", "").replace("Bearer ", "")
    # verify token here...
    return {
        "identity": "user123",
        "display_name": "John Doe",
        "permissions": ["read", "write"],
        "is_authenticated": True,
```

```
        # custom fields are preserved
        "role": "admin",
        "team_id": "team456",
    }
```

And here's where it gets interesting: handlers can **deny**, **filter**, or **modify** requests.

```
@auth.on.threads.search
async def filter_by_team(ctx, value):
    return {"metadata": {"team_id": ctx.user.team_id}}

@auth.on.threads.create
async def inject_team_id(ctx, value):
    value.setdefault("metadata", {})
    value["metadata"]["team_id"] = ctx.user.team_id
    return value

@auth.on.assistants.delete
async def restrict_deletion(ctx, value):
    if ctx.user.role != "admin":
        return False
```

> *Authorization handlers are non-interruptive by default (if no auth is configured >> requests allowed; if no handlers → allowed).*

This is both helpful and dangerous.

So here's my checklist :

- Do we have an auth handler configured in `aegra.json`?
- Do our handlers enforce tenant boundaries for threads/runs/store?
- Are we injecting metadata (team/user IDs) server-side, not trusting the client?
- Do we have a "break glass" admin path that is explicitly controlled?
- Do we have manual auth tests (even a tiny smoke suite)?

Custom routes are a feature

Aegra supports mounting your own FastAPI app for custom routes (`http.app`).

```
{
  "graphs": { "agent": "./graphs/react_agent/graph.py:graph" },
  "http": {
    "app": "./custom_routes.py:app",
    "enable_custom_route_auth": false,
    "cors": {
      "allow_origins": ["https://example.com"],
      "allow_credentials": true
    }
  }
}
```

And the route priority rules are the sort of detail you want documented:

1. Unshadowable routes (`/health`, `/ready`, `/live`, `/docs`, `/openapi.json`) always accessible
2. Your custom routes take precedence
3. Shadowable routes (`/`, `/info`) can be overridden
4. Protected core routes (`/assistants`, `/threads`, `/runs`, `/store`) cannot be overridden

The auth default is also explicit:

- custom routes do **not** automatically have Aegra auth applied

- you can enable auth for all custom routes with `enable_custom_route_auth: true`
- or you can apply auth per-route using a dependency

```
from fastapi import Depends
from aegra_api.core.auth_deps import require_auth

@app.get("/my-protected-route")
async def my_route(user = Depends(require_auth)):
    return {"user": user.identity}
```

Custom routes are where agent platform turns into product backend. Treat them like real endpoints, i.e. threat model, auth, rate limiting, the whole deal.

So you can self-host if:

- You need **data control** and "bring your own Postgres"
- You need **custom auth** (JWT/OAuth/Firebase/custom) with real authorization hooks
- You want **OTEL-first observability** and the freedom to choose tracing backends
- You expect to build custom endpoints around your agent runtime
- You're allergic to vendor lock-in (or your compliance team is)

Stick with a managed platform if:

- Your team can't reliably operate Postgres + migrations yet
- You want "someone else handles upgrades/backup/HA" more than you want control
- You don't need custom authZ beyond basics
- You're optimizing for speed of experimentation over infrastructure ownership

Concluding thoughts

I like Aegra's philosophy and the repo also contains signals of engineering maturity :

- a deliberate concurrency model for graph execution (cache compiled structure, isolate execution instances)
- explicit migration discipline (auto-migrate, plus a documented breaking DB upgrade path)
- OTEL-first tracing fan-out (avoid being forced into a single vendor)
- auth as a policy layer (filters + metadata injection + denial, not just "JWT on/off")
- semantic store as config, with clarity on dimensions, fields, and providers

But the biggest reason I'm paying attention is simpler:

> *It treats agent backend like infrastructure you can reason about.*

And that's what most teams need right now.

You can find [further details in the repository.]

If you're building an AI product, let me know your take

1. What's the *first* thing that broke when you tried to productionize an agent: state, auth, observability, cost, something else?

2. How are you handling multi-tenancy today, e.g. client-side scoping, server-side filters, or something more robust?
3. Would you rather be locked into a tracing vendor… or locked into operating OTEL correctly?

Bonus Articles

Local LLMs That Can Replace Claude Code

Small team of engineers can easily burn >$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you…

medium.com

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents

Fully Autonomous Companies: OpenClaw Gateway + Routing + Agents Whether you think it's hype or not, people are already…

medium.com

Observational Memory for Long-Running Agents - "This wasn't the plan!"

Agents have unbounded context growth problem. Tool calls can easily dump 50k tokens into your thread, and helpful…

medium.com

ClawRouter: Anthropic charged me $4,660 — How I cut it 70% with smart LLM routing

Last month I opened my credit-card statement and almost threw up. Anthropic charged me $4,660.87, just for AI APIs.

medium.com

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison

Codex 5.3 vs. Opus 4.6: One-shot Examples and Comparison Just after 9:45 a.m. Pacific on 5 February 2026, Anthropic…

medium.com

Clawdbot Lite: 99% Smaller, 4000 Lines, Same Core Power

People are building Polymarket bots, multi-agent product teams, crypto traders, coding agents, business operators with...

medium.com

Vectorless RAG for Agents: PageIndex Is Why Their Demo Works and Yours Needs “Context”

When I’m building a RAG apps, the thing that quietly kills quality (and budgets) is turning a document into a pile of...

medium.com
#langgraph#langgraph-agents#ai-agent#agentic-ai#self-hosted

Reporting a Problem

Sometimes we have problems displaying some Medium posts.

If you have a problem that some images aren't loading - try using VPN. Probably you have problem with access to Medium CDN (or fucking Cloudflare's bot detection algorithms are blocking you).

Problem Description Describe your problem here...

Submit Cancel